

Measurement Integrity for Trustworthy Agentic Systems: Failure Modes and a Reproducible Evaluation Architecture

Anonymous Submission

Abstract

Evaluating a multi-step agent is not one measurement but several, layered together: provider inference, shell and tool execution, a mutable environment, trajectory logging, checkpoint and replay, and an offline benchmark evaluator. Each boundary between these layers can fail silently, and a silent failure does not announce itself as an error — it produces a plausible number. We report five controlled failure cases from an instrumented coding-agent harness in which a defensible-looking measurement choice corrupts behavioural features, ground-truth task outcomes, or resource accounting. In one, omitting the benchmark’s official test patch makes the target test uncollectable, so the evaluator reports no test result at all; in another, a failing test piped into `tail` is recorded as a successful tool call. We then describe a reference architecture that separates online trajectory state, provider telemetry, and offline evaluation behind a one-way boundary, and an artifact that enforces nine validity invariants as regression tests. Our results are preliminary and methodological: they demonstrate that these failures are possible and cheaply preventable, not how often they occur.

1 Introduction

If an agent benchmark cannot distinguish an agent failure from an infrastructure failure, its reliability estimate is itself unreliable. Trustworthy agentic systems are increasingly justified by quantitative claims — success rates, failure predictors, early-abort policies, routing decisions — and every such claim inherits the validity of the measurement system that produced it.

Multi-step agents make that inheritance unusually fragile. A single-turn benchmark measures one model output against one reference. An agentic evaluation instead threads *model* $\rightarrow$ *tool* $\rightarrow$ *environment* $\rightarrow$ *model* $\rightarrow$ *evaluation*, repeatedly, through components that were never designed to be a measurement apparatus: a shell, a container filesystem, a rate-limited HTTP API, and a test runner. Each hand-off is a place where the quantity being recorded can drift from the quantity being claimed. The drift is rarely loud. A shell pipeline returns 0; a file-change counter returns a large number; an evaluator returns “no failures.” None of these looks like a bug.

This paper is a measurement-integrity study rather than an agent-performance study. We make no claim about any model’s or scaffold’s capability. We instead treat the evaluation harness itself as the object under test, and ask what has to be true for its outputs to support trustworthy conclusions.

Contributions.

- **Controlled demonstrations** of five measurement-integrity failures in agentic coding evaluation, each reproduced from a released artifact with no paid API access (Section 3).
- **A reference measurement architecture** separating online trajectory state, provider/infrastructure telemetry, and offline evaluation, with a one-way boundary that prevents outcome and future information from reaching online features (Section 4).
- **A reproducible artifact** implementing nine validity invariants as regression tests, with a single deterministic command that regenerates every number we report (Section 4).

2 A Measurement Integrity Model

We model an agentic evaluation as three channels that must not leak into one another.

Online trajectory (O_t). Everything knowable at step t : issued commands, exit statuses, output sizes, token counts, repository state. Online behavioural features — the inputs to any deployable monitor or early-abort policy — must be computable from O_t alone. If a feature can see step $t+1$ or the final outcome, a reported “early prediction” is partly a readout of the answer.

Provider telemetry (P). Physical API attempts, HTTP 503s, timeouts, quota events, and infrastructure termination reasons. These describe the *service*, not the agent. A retried 503 is not a reasoning error, and a run killed by a quota limit is not a task failure.

Offline evaluation (E). The benchmark verdict Y , computed in a clean state from official test data after the agent has stopped. Y must never be visible to O_t .

Two directed constraints follow, and they are the whole design: $E \nleftrightarrow O_t$ (no outcome leakage) and $P \nleftrightarrow O_t$ as *semantics* (infrastructure events may be logged, but may not become agent behaviour). Figure 1 shows the resulting architecture.

3 Failure Cases

We instrumented a coding-agent harness (mini-SWE-agent scaffold, SWE-bench Verified instances, containerised execution) and recorded cases where a natural implementation choice silently corrupts a measured quantity. Table 1 summarises them. All values are regenerated by the command in Section 4; E1–E3 use a local container and no model inference, E4–E5 are pure Python.

E1: Omitted official test patch. SWE-bench `FAIL_TO_PASS` identifiers frequently name test parametrisations that exist only *after* the instance’s official `test_patch` is applied. Running the evaluator without it, on `matplotlib__matplotlib-23412`, yields pytest exit code 4 with “no tests ran” and “ERROR: not found”: there is no test result at all. With the patch applied, the same command yields

exit code **1** and **1 failed** — a genuine signal. Both arms return `success = false`, so a harness checking only a success flag sees nothing wrong; but the first arm’s “0 failures” is vacuous, and any pipeline that reads absence-of-failure as passing will emit false successes.

E2: Pipeline exit-code masking. Agents pipe constantly (`pytest ... | tail -5`). Without `set -o pipefail` a POSIX shell reports the status of the *last* stage. Measured in-container: `false | tail -5` returns **1** with `pipefail` and **0** without; the same holds for a failing Python invocation. Two of three probe commands have their failure fully masked. Every downstream failure-rate feature is then biased toward zero exactly when the agent is struggling.

E3: Container-wide change attribution. Counting modified files with `docker diff` attributes the whole container to the agent. In an earlier run on a real instance this reported **537** changed files at step 3, essentially all conda and `pytest-cache` churn. We reproduce the mechanism directly: executing a command that touches *no* repository file moves the container-wide count from **0** to **32** while the repository-scoped git measurement stays at **0**. A single one-line source edit then moves the container count to **33** — indistinguishable from

noise — while the git measurement moves to exactly **1 file, 2 lines**. The naive counter cannot separate agent edits from environment churn.

E4: Oracle and future leakage. We compute online features from a length- k prefix, then append 20 maximally predictive future steps (exit status 127, `FATAL` output, 10^6 tokens). Across **24** online features, **0** change. Reading a step at or beyond the cut raises `PrefixBoundaryError`, and the prefix object exposes no outcome attribute. This is an adversarial regression test, not a proof: it demonstrates that the leak path is closed by construction rather than by convention.

E5: Infrastructure-to-label contamination. Provider failures must not enter the label set. Enumerating all termination reasons, the three classified as provider infrastructure (unavailable, timeout, rate-limited) are structurally refused a Y label by a schema validator, while the five agent-semantic reasons accept one; there are **0** violations. In our own runs this was not hypothetical: **8** recorded sessions terminated on provider quota exhaustion, and all 8 carry no Y label. Had they been scored, they would have entered a dataset as fabricated task failures.

Failure mode	Naive measurement	Observed consequence	Corrected invariant
E1 Missing evaluator test patch	Run <code>FAIL_TO_PASS</code> ids against the repo as-is	<code>pytest</code> exit 4, “no tests ran”; no test result exists, yet “0 failures” is reportable	Reset test files to <code>base_commit</code> and apply the official <code>test_patch</code> before evaluating (I3)
E2 Pipeline exit-code masking	Record the shell’s exit status for a piped command	<code>false tail -5</code> → 0 instead of 1; failure-rate features biased toward zero	Execute every agent command under <code>set -o pipefail</code>
E3 Container-wide file counting	<code>docker diff</code> line count	Pure env churn moves the count 0→32 while repo-scoped stays 0 (537 in an earlier real run)	Repository-scoped git state with an explicit measured flag (I6)
E4 Oracle / future leakage	Compute features over the full trajectory object	Future steps and Y reachable from “online” features; 20 adversarial future steps change 0 of 24	Features accept only a length- k prefix copy exposing no outcome (I1, I2)
E5 Provider error as task failure	Any non-completing run scores $Y=0$	8 quota-terminated sessions would have become fabricated failures	Infrastructure terminations enumerated in one set; validator refuses any label (I4, I5)

Table 1: Five measurement-integrity failure modes. Every number is regenerated by `scripts/paper/reproduce_all.py` into `results/paper/`.

4 Reference Architecture and Artifact

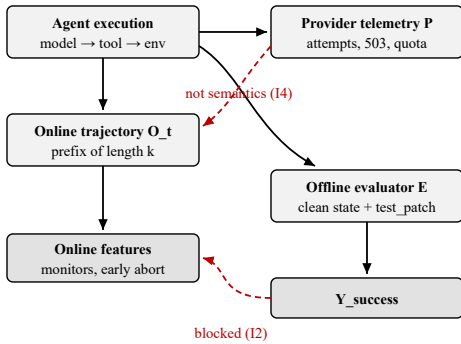


Figure 1: Reference architecture. Agent execution feeds the online trajectory, provider telemetry, and (after termination) the offline evaluator. The two dashed edges are the ones that must not exist: benchmark outcome must never reach online features, and provider events must never become agent semantics.

The architecture instantiates Section 2 as three separated channels. Online feature extraction is typed to accept only a `TrajectoryPrefix` — an immutable copy of the first k steps that holds no outcome record — so leakage is a type error rather than a code-review question. Provider retries and physical attempts are handled strictly below trajectory semantics, so a recovered 503 is invisible to behavioural features while remaining fully logged. The evaluator runs after termination in a clean state, resetting test files to the instance base commit before applying the official test patch, so the agent cannot influence the tests that judge it.

Reference invariants. Our artifact enforces the following as regression tests. We present these as the invariants *this* artifact implements, not as a claim that they are universally necessary or sufficient.

- I1** Online prefix cannot access future trajectory information. **I2** Online prefix cannot access Y_{success} . **I3** Evaluation state is isolated from agent execution state. **I4** Provider failures do not increment semantic failure features. **I5** Infrastructure-invalid runs receive no Y_{success} . **I6** Repository-change features are repository-scoped. **I7** Raw trajectory artifacts are immutable and hashed. **I8** Logical agent

calls are distinct from physical provider attempts. **19** Restored continuations receive new identities and cannot overwrite parents.

The artifact ships 128 passing tests covering these invariants, including adversarial cases (E4), negative cases for test-command detection, and provider-error classification. Every number in this paper is regenerated by one command requiring no paid API:

```
uv run python scripts/paper/reproduce_all.py
```

which writes machine-readable results for E1–E5 to `results/paper/`. E4 and E5 run anywhere; E1–E3 additionally require a local container runtime and one benchmark image.

5 Discussion and Limitations

Limitations. (1) Our failure cases demonstrate *possibility and mechanism*, not *prevalence*; we make no estimate of how often these defects occur in published harnesses. (2) The artifact is validated primarily for mini-SWE-agent / SWE-bench-style coding-agent workflows, and the invariants require evaluation on other agent frameworks before being treated as general. (3) The provider events we report are case-study infrastructure failures encountered on one free-tier account; they are *not* estimates of any provider's service reliability. (4) We have not established real-provider replay stability: our same-condition continuations were terminated by quota limits before producing valid comparisons, so we report no replay-fidelity result. (5) We report no predictor, scheduler, or large-scale empirical study, and no claim about agent success rates. (6) Several of our own numbers were produced by fixing defects we introduced, which is evidence that the invariants are testable, not that they are complete.

Questions for the community. These cases suggest that trustworthy-agent benchmarking itself needs assurance practices, and we would like TAS discussion on four questions. *What constitutes a valid agent failure?* A run killed by a quota limit, a container timeout, and a genuine reasoning error are currently often scored identically. *How should infrastructure-invalid executions be reported?* Silently dropping them biases success rates upward; scoring them as failures biases them downward; we retain them immutably and label them as invalid. *What may an online evaluation system observe?* Claims about early failure prediction are only meaningful relative to a stated information boundary. *What provenance should agent benchmark artifacts record?* We argue for physical-attempt ledgers, hashed raw trajectories, and explicit run dispositions as a minimum.

6 Related Work

Recent work characterises *how* agents fail and how early failure can be detected. Zhao et al. (2026) analyse CLI coding-agent failure as a temporal process across thousands of trajectories; Ruan et al. (2026) predict eventual failure from early rounds to abort doomed episodes; and Li et al. (2026) derive online warning signals for wasted computation in multi-agent systems. Each of these depends on the trajectory record

being a faithful account of what happened — exactly the property we test. Gonuguntla (2026) shows that statically replaying logged trajectories under a substituted model scores a counterfactual world, and uses branching rollouts instead; that argument is about the validity of an evaluation procedure, and is close in spirit to ours, though it targets model switching rather than instrumentation boundaries. SWE-bench (Jimenez et al. 2024) and its human-validated subset (OpenAI 2024) supply the task and evaluator we instrument, and SWE-agent (Yang et al. 2024) established the agent-computer interface such harnesses build on. Kapoor et al. (2024) argue that agent benchmarking is beset by methodological problems including inadequate holdout and cost-blind comparison; our contribution is complementary and narrower, targeting validity at the boundary between agent execution, provider infrastructure, instrumentation, and offline evaluation. We do not claim to be first to observe any individual defect; we claim value in demonstrating them concretely and closing them with enforced invariants.

7 Conclusion

Trustworthy agentic systems require trustworthy measurements, and in multi-step evaluation the measurement apparatus is itself a distributed system with several failure-prone boundaries. We demonstrated five controlled cases where a reasonable implementation choice silently corrupts a behavioural feature, a task label, or a resource count, and we described an architecture that separates online trajectory state, provider telemetry, and offline evaluation behind a one-way boundary, enforced by nine regression-tested invariants. The results are preliminary and methodological. We offer them less as a solution than as a concrete starting point for a shared discussion about what an agent evaluation harness must guarantee before its numbers can carry weight.

References

- Gonuguntla, A. 2026. The Replay Gap: Static Evaluation of Model Switching in LLM Agents Scores the Wrong World. *arXiv:2608.08239*.
- Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *ICLR*.
- Kapoor, S.; Stroebl, B.; Siegel, Z. S.; Nadgir, N.; and Narayanan, A. 2024. AI Agents That Matter. *arXiv:2407.01502*.
- Li, X.; Yan, W.; Wu, Y.; Liang, P.; Yuan, M.; Liu, J.; and Yang, J. 2026. Early Diagnosis of Wasted Computation in Multi-Agent LLM Systems via Failure-Aware Observability. *arXiv:2606.01365*.
- OpenAI. 2024. Introducing SWE-bench Verified. openai.com/index/introducing-swe-bench-verified/.
- Ruan, K.; Huang, Z.; Zhou, Z.; Wei, Q.; Wang, X.; and Sun, H. 2026. Doomed from the Start: Early Abort of LLM Agent Episodes via a Recall-Controlled Probe Cascade. *arXiv:2607.06503*.
- Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *NeurIPS*.
- Zhao, X.; Li, H.; Li, S.; Zhao, T.; Barr, E. T.; Sarro, F.; and Ye, H. 2026. Failure as a Process: An Anatomy of CLI Coding Agent Trajectories. *arXiv:2607.09510*.